OVER THE WIRE

BANDIT WARGAME REPORT

Levels 0–21

Student: **Huseyn Rahimov**

Course / Group: **Millisec.edu.az**

Date: **26-Oct-26**

Teacher: **Zaur Huseynov**

Contents

1. Introduction.....
2. Kali Linux Overview.....
3. OverTheWire Bandit Overview.....
4. Practical Work: Level 0.....
5. Practical Work: Level 1.....
6. Practical Work: Level 2.....
7. Practical Work: Level 3.....
8. Practical Work: Level 4.....
9. Practical Work: Level 5.....
10. Practical Work: Level 6.....
11. Practical Work: Level 7.....
12. Practical Work: Level 8.....
13. Practical Work: Level 9.....
14. Practical Work: Level 10.....
15. Practical Work: Level 11.....
16. Practical Work: Level 12.....
17. Practical Work: Level 13.....
18. Practical Work: Level 14.....
19. Practical Work: Level 15.....
20. Practical Work: Level 16.....
21. Practical Work: Level 17.....
22. Practical Work: Level 18.....
23. Practical Work: Level 19.....
24. Practical Work: Level 20.....
25. Practical Work: Level 21.....
26. Skills Developed.....
27. Conclusion.....

1. Introduction

This report documents practical work with the OverTheWire Bandit wargame, a beginner-friendly environment for learning Linux command-line skills and basic security concepts. The exercises progress from connecting to a remote machine and reading files to searching directories, decoding data, using SSH keys, communicating with network services, and understanding scheduled tasks.

The report is organized as a learning record. Each level includes its goal, commands, an explanation of the method, a blank password field, and a placeholder for a screenshot. Password fields are intentionally left empty so that the student can enter the values obtained during their own lab session. Screenshots should show the relevant command and result without exposing unrelated personal information.

2. Kali Linux Overview

Kali Linux is a Debian-based Linux distribution maintained for penetration testing, digital forensics, security auditing, and cybersecurity education. It includes a large collection of tools for network analysis, web testing, password auditing, and incident investigation. Kali can run on a physical computer, in a virtual machine, or as a live system.

2.1 Working in the terminal

The terminal provides a text interface for running commands. Commands are interpreted by a shell, which also supports paths, variables, pipes, redirection, and command chaining. Learning to use the terminal is important because many security tools and remote Linux servers are managed through command-line interfaces.

```
pwd        # print the current working directory
ls -la     # list files, including hidden files, with details
cd folder  # change to a directory
cat file   # print a text file
man find   # open the manual page for find
```

2.2 File permissions and users

Linux associates files with an owner and a group. Permission bits define whether the owner, group, and other users may read, write, or execute a file. The commands ls -l, id, and find can help inspect ownership and permissions. Administrative access should be used carefully, and lab tasks should be performed only on systems where permission has been granted.

2.3 Responsible security practice

Tools and techniques should be used only in authorized environments. A training wargame such as Bandit provides a controlled place to practice. The same commands should not be used to access systems, accounts, or data without permission.

3. OverTheWire Bandit Overview

Bandit is an online wargame from OverTheWire designed to teach command-line basics through a sequence of small challenges. Players connect to the Bandit server using SSH. Each level provides a task, and the password for the next account is usually discovered by solving that task. The official game instructions identify the server, username, and port used for the connection.

3.1 Connecting to Bandit

```
ssh bandit0@bandit.labs.overthewire.org -p 2220
```

The command starts an SSH session to the Bandit host on port 2220. The username selects the account. At the first connection, the SSH client may ask whether to trust the server host key; verify prompts according to the lab instructions before accepting. After login, use the level-specific clues to find the next password.

3.2 A repeatable approach

1. Read the level instructions carefully.
2. Inspect the current directory with pwd and ls -la.
3. Use file, find, grep, strings, sort, or other tools indicated by the clue.
4. Read command help or manual pages when options are unfamiliar.
5. Record the method and result, but keep passwords private.
6. Capture a screenshot showing the relevant evidence for the report.

4. Practical Work: Levels 0–21

The procedures below describe the standard Bandit level transitions. Replace the blank password lines with the values obtained in the authorized lab. Commands may need small adjustments depending on the current directory and the exact output shown by the server.

Level 0: Connecting with SSH

Objective: Establish the first SSH connection to the Bandit server.

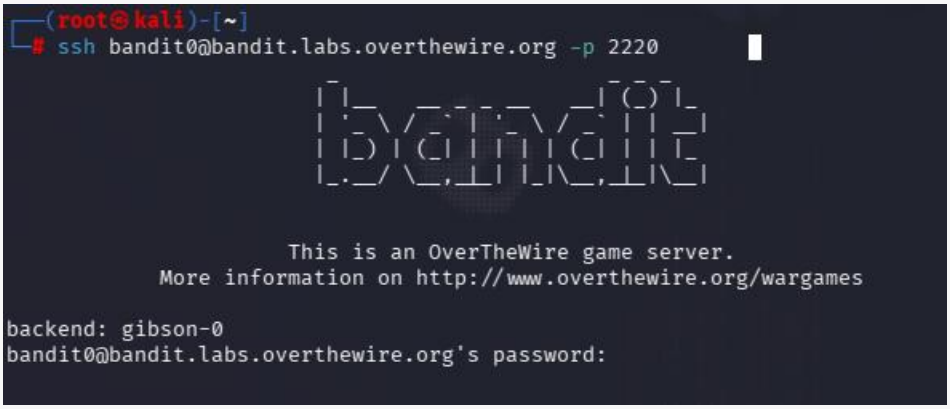
Commands / procedure:

```
ssh bandit0@bandit.labs.overthewire.org -p 2220
```

Explanation: SSH opens an encrypted remote terminal session. The -p option specifies port 2220, which is used by the Bandit game rather than the default SSH port.

Important note: This level is primarily about connecting; the initial credentials are provided by the official game page.

Password for bandit0: **bandit0**



Level 1: Reading a File

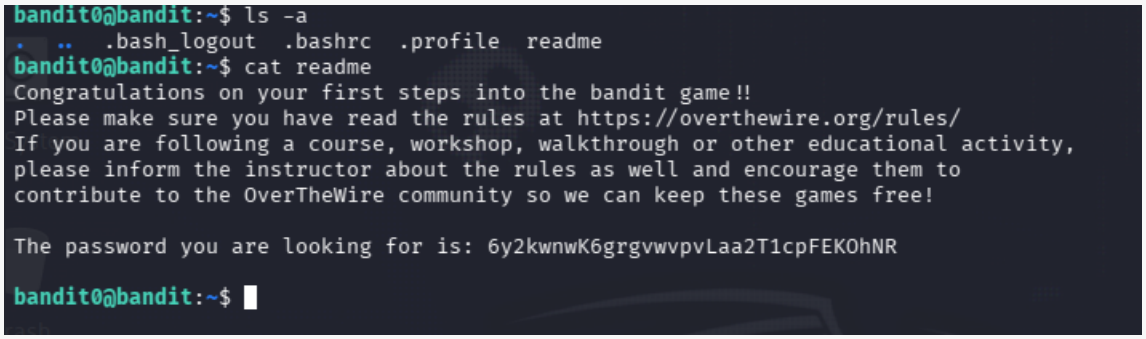
Objective: Find the password stored in the file named readme.

Commands / procedure:

```
ls -la
cat readme
```

Explanation: The ls command lists files in the current directory. cat displays the contents of readme, which contains the password for the next account.

Password for bandit1: 6y2kwnwK6grgvwvpvLaa2T1cpFEKOhNR



Level 2: Opening a Filename Beginning with a Dash

Objective: Read a file whose name is a single dash (-).

Commands / procedure:

```
ls -la
cat ./-
```

Explanation: A leading dash is often interpreted as a command-line option. Prefixing the filename with ./ makes it an explicit path relative to the current directory, so it is treated as a filename.

Password for bandit2: 263JGJPfgU6LtdEvgfWU1XP5yac29mFx

```
bandit1@bandit:~$ ls -al
total 24
-rw-r-----   1 bandit2 bandit1   33 Oct 14 09:26 -
drwxr-xr-x   2 root    root    4096 Oct 14 09:26 .
drwxr-xr-x 150 root    root    4096 Oct 14 09:29 ..
-rw-r--r--   1 root    root     220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root    root    3851 Oct 14 09:19 .bashrc
-rw-r--r--   1 root    root     807 Mar 31  2024 .profile
bandit1@bandit:~$ cat ./-
263JGJPfgU6LtdEvgfWU1XP5yac29mFx
```

Level 3: Handling Spaces in a Filename

Objective: Read a file whose name contains spaces.

Commands / procedure:

```
ls -la
cat "spaces in this filename"
```

Explanation: Shells use spaces to separate command arguments. Quotation marks group the filename into one argument. Escaping each space with a backslash is another valid approach.

Password for bandit3: MNk8KNH3Usiio41PRUEoDFPqfxLPlSmx

```
bandit2@bandit:~$ ls
--spaces in this filename--
bandit2@bandit:~$ cat ./"--spaces in this filename--
MNk8KNH3Usiio41PRUEoDFPqfxLPlSmx
bandit2@bandit:~$
```

Level 4: Finding a Hidden File

Objective: Locate a hidden file in the inhere directory.

Commands / procedure:

```
ls -la
cd inhere
ls -la
cat .hidden
```

Explanation: On Linux, filenames beginning with a dot are hidden from ordinary ls output. The -a option includes hidden entries, allowing the relevant file to be identified and read.

Password for bandit4: 2WmrDFRmJIq3IPxneAaMGhap0pFhF3NJ

```
bandit3@bandit:~$ ls
inhere
bandit3@bandit:~$ cd inhere/
bandit3@bandit:~/inhere$ ls -al
total 12
drwxr-xr-x 2 root    root    4096 Oct 14 09:26 .
drwxr-xr-x 3 root    root    4096 Oct 14 09:26 ..
-rw-r----- 1 bandit4 bandit3   33 Oct 14 09:26 ...Hiding-From-Y
bandit3@bandit:~/inhere$ cat ...Hiding-From-You
2WmrDFRmJIq3IPxneAaMGhap0pFhF3NJ
bandit3@bandit:~/inhere$
```

Level 5: Identifying Human-Readable Data

Objective: Find the human-readable file among the files in inhere.

Commands / procedure:

```
cd inhere
ls -la
file ./*
cat ./<matching-file>
```

Explanation: The file command examines file contents and reports their likely type. Reviewing the output helps identify the human-readable file. Replace <matching-file> with the actual filename found during the exercise.

Important note: Do not type the angle-bracket placeholder literally; substitute the filename.

Password for bandit5: 4oQYVPkxZ0OE005pTW81FB8j8lxXGUQw

```
bandit4@bandit:~$ ls -al
total 24
drwxr-xr-x   3 root root 4096 Oct 14 09:26 .
drwxr-xr-x 150 root root 4096 Oct 14 09:29 ..
-rw-r--r--   1 root root  220 Mar 31  2024 .bash_logou
-rw-r--r--   1 root root 3851 Oct 14 09:19 .bashrc
drwxr-xr-x   2 root root 4096 Oct 14 09:26 inhere
-rw-r--r--   1 root root  807 Mar 31  2024 .profile
bandit4@bandit:~$ cd inhere/
bandit4@bandit:~/inhere$ ls -al
total 48
drwxr-xr-x 2 root    root    4096 Oct 14 09:26 .
drwxr-xr-x 3 root    root    4096 Oct 14 09:26 ..
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file00
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file01
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file02
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file03
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file04
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file05
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file06
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file07
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file08
-rw-r----- 1 bandit5 bandit4   33 Oct 14 09:26 -file09
bandit4@bandit:~/inhere$ file ./-file0*
./-file00: data
./-file01: OpenPGP Public Key
./-file02: OpenPGP Public Key
./-file03: data
./-file04: data
./-file05: data
./-file06: data
./-file07: ASCII text
./-file08: data
./-file09: data
bandit4@bandit:~/inhere$ cat ./-file07
4oQYVPkxZOOEOO5pTW81FB8j8lxXGUQw
bandit4@bandit:~/inhere$
```

Level 6: Searching by File Properties

Objective: Find a file of the required size and permissions in the inhere directory.

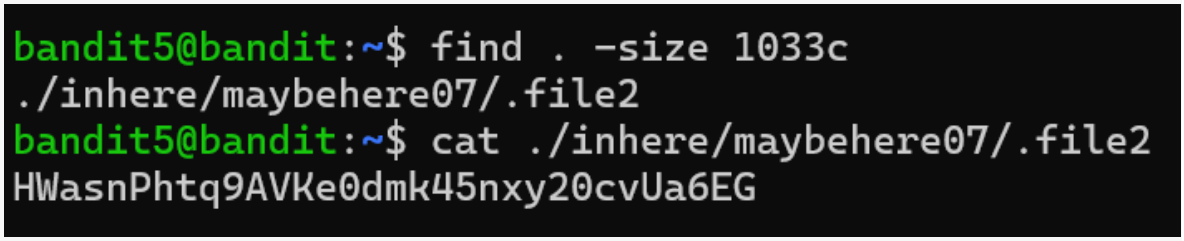
Commands / procedure:

```
cd inhere
find . -type f -size 1033c
file ./<matching-file>
cat ./<matching-file>
```

Explanation: find searches recursively. The -type f option limits results to regular files, and -size 1033c specifies exactly 1033 bytes. Verify the candidate with file and read it.

Important note: The official clue includes additional conditions such as being non-executable and human-readable; inspect candidates if more than one result appears.

Password for bandit6: HWasnPhtq9AVKe0dmk45nxy20cvUa6EG



Level 7: Searching by Owner, Group, and Size

Objective: Find the file owned by user bandit7, group bandit6, and 33 bytes in size.

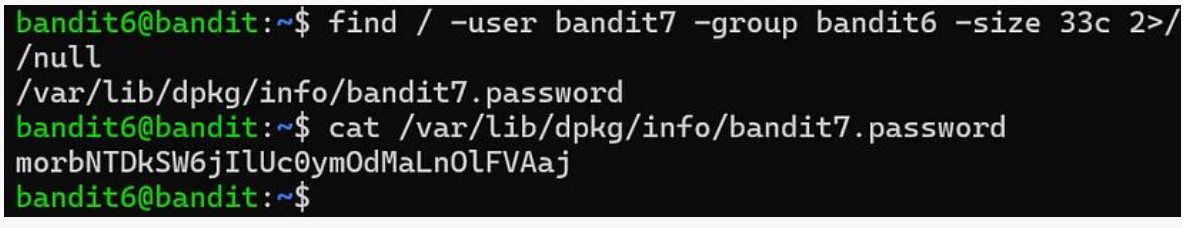
Commands / procedure:

```
find / -type f -user bandit7 -group bandit6 -size 33c 2>/dev/null
cat <matching-path>
```

Explanation: Searching from / checks the filesystem. The -user, -group, and -size options filter by ownership and size. Redirecting stderr to /dev/null suppresses permission-denied messages; read the matching file using its full path.

Important note: Use only the path returned by find; some directories will not be accessible.

Password for bandit7: morbNTDkSW6jIlUc0ymOdMaLnOlFVAaj



Level 8: Finding a Word in a Large File

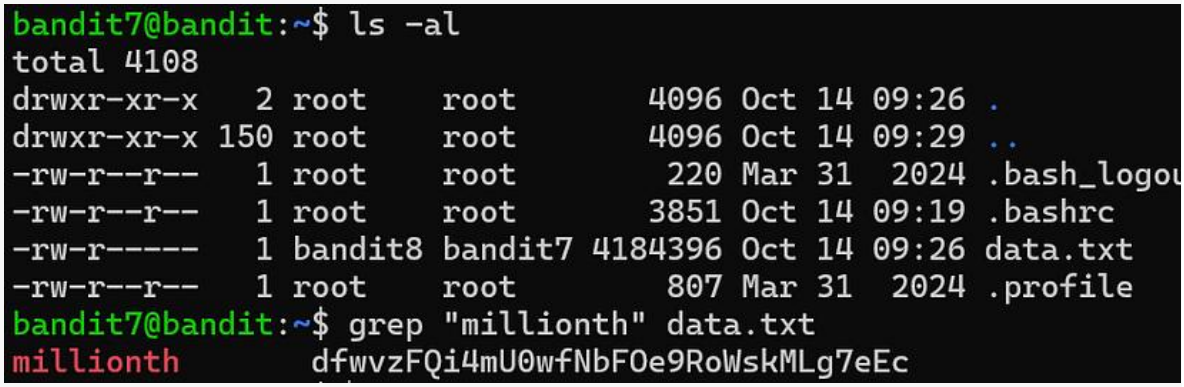
Objective: Find the password associated with the word millionth in data.txt.

Commands / procedure:

```
grep "millionth" data.txt
```

Explanation: grep searches text for lines matching a pattern. The matching line contains the target word and the value needed for the next level.

Password for bandit8: dfwvzFQi4mU0wfNbFOe9RoWskMLg7eEc



Level 9: Finding a Unique Line

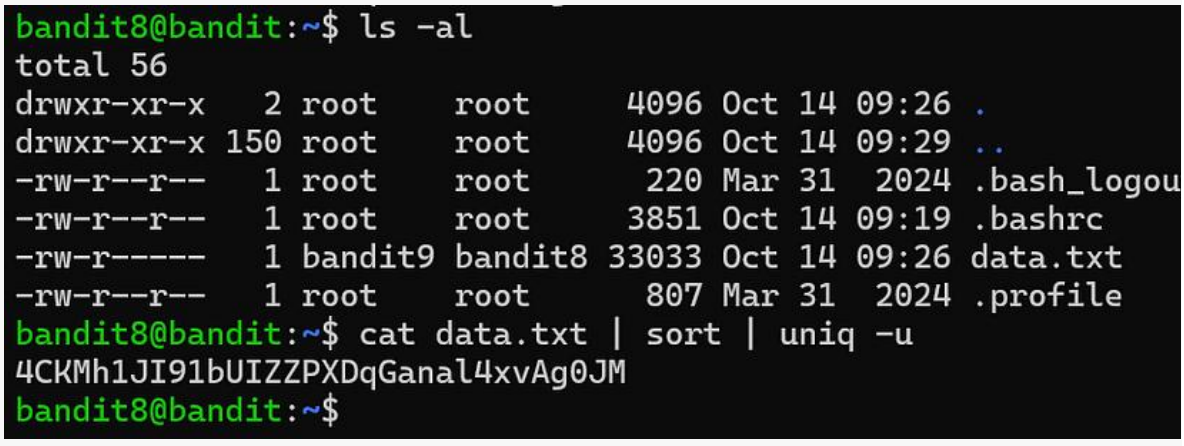
Objective: Identify the line that occurs only once in data.txt.

Commands / procedure:

```
sort data.txt | uniq -u
```

Explanation: sort groups identical lines together. uniq -u prints lines that occur only once, so the combination isolates the unique entry. The pipe passes the output of sort to uniq.

Password for bandit9: 4CKMh1JI91bUIZZPXDqGanal4xvAg0JM



Level 10: Extracting Readable Strings

Objective: Find the human-readable password in data.txt among binary data.

Commands / procedure:

```
strings data.txt
strings data.txt | grep "==="
```

Explanation: strings extracts printable text sequences from binary or mixed-content files. Filtering the output around the marker shown in the challenge can make the relevant line easier to spot.

Important note: If the marker does not match the file output, inspect the strings output and use the marker shown by the challenge.

Password for bandit10: FGUW5ilLVJrxX9kMYMmlN4MgbpfMiqey

```
bandit9@bandit:~$ strings data.txt | grep "^=="
========== the
========== password
========== FGUW5ilLVJrxX9kMYMmlN4MgbpfMiqey
```

Level 11: Decoding Base64

Objective: Decode the Base64-encoded content in data.txt.

Commands / procedure:

```
cat data.txt
base64 -d data.txt
```

Explanation: Base64 represents binary data using printable characters. The -d option decodes the input and prints the decoded content, which contains the next password.

Password for bandit11: dtR173fZKb0RRsDFSGSg2RWnpNVj3qRr

```
bandit10@bandit:~$ ls -al
total 24
drwxr-xr-x   2 root     root     4096 Oct 14 09:26 .
drwxr-xr-x 150 root     root     4096 Oct 14 09:29 ..
-rw-r--r--   1 root     root      220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root     root     3851 Oct 14 09:19 .bashrc
-rw-r-----   1 bandit11 bandit10   69 Oct 14 09:26 data.txt
-rw-r--r--   1 root     root      807 Mar 31  2024 .profile
bandit10@bandit:~$ cat data.txt | base64 -d
The password is dtR173fZKb0RRsDFSGsg2RWnpNVj3qRr
```

Level 12: Applying a ROT13 Transformation

Objective: Reverse the ROT13 letter substitution used in data.txt.

Commands / procedure:

```
cat data.txt
tr 'A-Za-z' 'N-ZA-Mn-za-m' < data.txt
```

Explanation: ROT13 replaces each letter with the letter 13 positions later in the alphabet. Applying the same transformation again reverses it. tr performs the character substitution.

Password for bandit12: 7x16WNeHIi5YkIhWsfFIqoognUTyj9Q4

```
bandit11@bandit:~$ ls -al
total 24
drwxr-xr-x   2 root     root     4096 Oct 14 09:26 .
drwxr-xr-x 150 root     root     4096 Oct 14 09:29 ..
-rw-r--r--   1 root     root      220 Mar 31  2024 .bash_logo
-rw-r--r--   1 root     root     3851 Oct 14 09:19 .bashrc
-rw-r-----   1 bandit12 bandit11   49 Oct 14 09:26 data.txt
-rw-r--r--   1 root     root      807 Mar 31  2024 .profile
bandit11@bandit:~$ cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'
The password is 7x16WNeHIi5YkIhWsfFIqoognUTyj9Q4
```

Level 13: Unpacking a Nested Data File

Objective: Reverse the hex dump and unpack repeatedly until the readable password file is recovered.

Commands / procedure:

```
xxd -r data.txt > /tmp/data.bin
file /tmp/data.bin
mv /tmp/data.bin /tmp/data
file /tmp/data
gzip -d /tmp/data  # if identified as gzip; adjust filename
bzip2 -d /tmp/data  # if identified as bzip2; adjust filename
tar -xf /tmp/data -C /tmp/  # if identified as tar
```

Explanation: The challenge file is encoded as a hex dump and then compressed or archived multiple times. xxd -r reverses the hex dump. file identifies each layer; the matching decompression or archive tool removes it. Repeat the identify-and-extract process until a text file containing the password is found.

Important note: Use a separate temporary filename at each stage to avoid overwriting data. Choose the extraction command based on the output of file.

Password for bandit13: _____

[INSERT SCREENSHOT FOR LEVEL 13 HERE]

Level 14: Using an SSH Private Key

Objective: Use the supplied private key to log in as bandit14.

Commands / procedure:

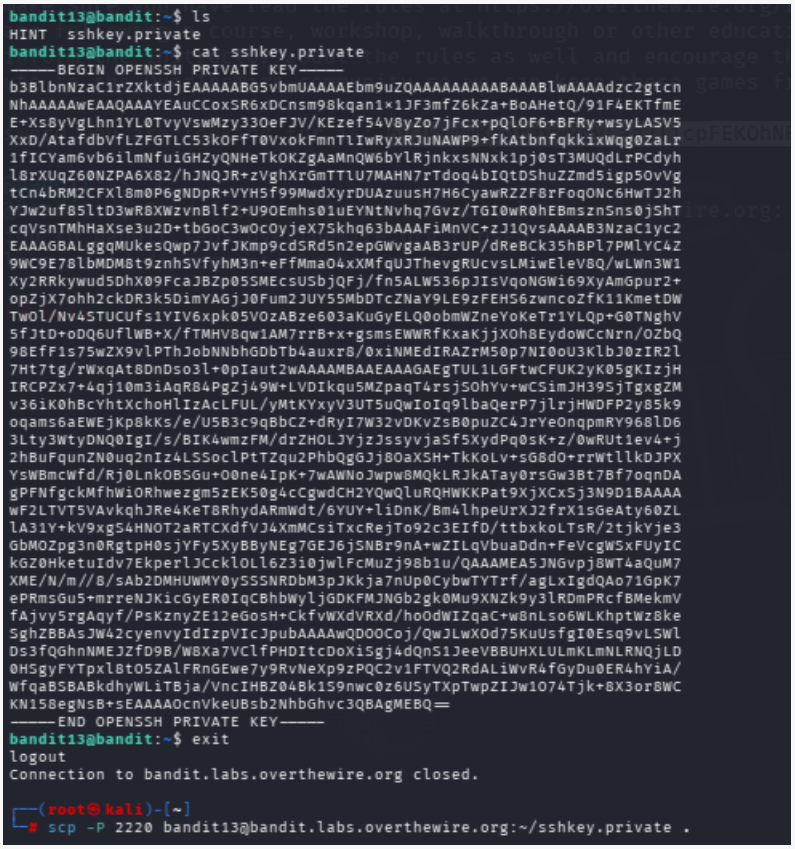
```
ls -la
ssh -i sshkey.private bandit14@localhost -p 2220
```

Explanation: The -i option tells SSH which private-key file to use for authentication. localhost refers to the same remote machine, and the specified port remains 2220. After logging in, the next challenge asks for the current password to be sent to a local service.

Important note: Protect private keys and do not share them outside the authorized lab.

Password for bandit14: _____

[INSERT SCREENSHOT FOR LEVEL 14 HERE



]

Level 15: Sending a Password to a Local Port

Objective: Send the current password to the service listening on localhost port 30000.

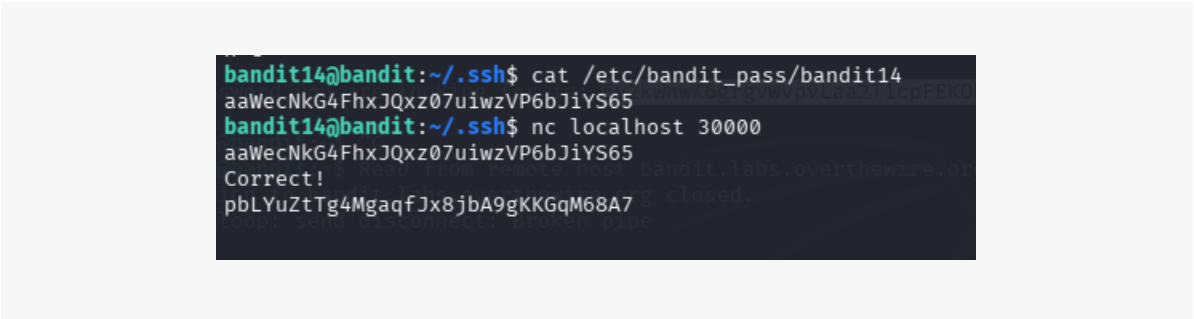
Commands / procedure:

```
cat /etc/bandit_pass/bandit14
nc localhost 30000
```

Explanation: The netcat client opens a TCP connection to the local service. Enter the current Bandit14 password when prompted; the service validates it and returns the next password.

Important note: Use the password for the current account, not the password you are trying to retrieve.

Password for bandit15: pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7



Level 16: Communicating over TLS

Objective: Connect securely to localhost port 30001 and submit the current password.

Commands / procedure:

```
openssl s_client -connect localhost:30001 -quiet
cat /etc/bandit_pass/bandit15
```

Explanation: openssl s_client establishes a TLS connection. The -quiet option reduces extra output. After the connection is established, send the current password through the connection to receive the next one.

Important note: The password should be entered into the TLS session after it connects; the cat command is shown as a way to inspect the current password if needed.

Password for bandit16: kS0Hf0u5HiXFwKMKFqXvPdOTNGGa0X8V

```
bandit15@bandit:~$ ls
bandit15@bandit:~$ ls -a
.  ..  .bandit14.password  .bash_logout  .bashrc  .profile
bandit15@bandit:~$ pwd
/home/bandit15
bandit15@bandit:~$ cat .bandit14.password
aaWecNkG4FhxJQxz07uiwzVP6bJiYS65
bandit15@bandit:~$ openssl s_client -connect localhost:30001
Connecting to 127.0.0.1
CONNECTED(00000003)
Can't use SSL_get_servername
depth=0 CN=SnakeOil
verify error:num=18:self-signed certificate
verify return:1
depth=0 CN=SnakeOil
verify return:1
---
Certificate chain
 0 s:CN=SnakeOil
   i:CN=SnakeOil
   a:PKEY: RSA, 4096 (bit); sigalg: sha256WithRSAEncryption
   v:NotBefore: Jun 10 03:59:50 2024 GMT; NotAfter: Jun  8 03:59:50 2034
---
Server certificate
-----BEGIN CERTIFICATE-----
MIIFBzCCAu+gAwIBAgIUBLz7DBxA0IfojaL/WaJzE6Sbz7cwDQYJKoZIhvcNAQEL
BQAwEzERMA8GA1UEAwwIU25ha2VPaWwwHhcNMjQwNjEwMDM1OTUwWhcNMzQwNjA4
MDM1OTUwWjATMREwDwYDVQQDDAhTbmFrZU9pbDCCAiIwDQYJKoZIhvcNAQEBBQAD
ggIPADCCAgoCggIBANI+P5QXm9Bj21FIPsQqbqZRb5XmSZZJYaam7EIJ16Fxedf+
jXAv4d/FVqiEM4BuSNsNMeBMx2Gq0lAfN33h+RMTjRoMb8yBsZsC063MLfXCk4p+
09gtGP7BS6Iy5XdmfY/fPHvA3JDEScdlDDmd6Lsbdwhv93Q8M6POVO9sv4HuS4t/
jEjr+NhE+Bjr/wDbyg7GL71BP1WPZpQnRE4OzoSrt5+bZVLvODWUFwinB0fLaGRk
GmI0r5EUOUd7HpYyoIQbiNlePGfPpHRKnmdXTTEZEoxeWWAaM1VhPGqfrB/Pnca+
vAJX7iBOb3kHinmfVOScsG/YAUR94wSELeY+UlEWJaELVUntrJ5HeRDiTChiVQ++
wnnjNbepaW6shopybUF3XXfhIb4NvwLWpvoKFXVtcVjlOujF0snVvpE+MRT0wacy
tHtjZs7Ao7GYxDz6H8AdBLKJW67uQon37a4MI260ADFMS+2vEAbNSFP+f6ii5mrB
18cY64ZaF6oU8bjGK7BArDx56bRc3WFyuBIGWAFHEuB948BcshXY7baf5jjzPmgz
mq1zdRthQB31MOM2ii6vuTkheAvKfFf+llH4M9SnES4NSF2hj9NnHga9V08wfhYc
x0W6qu+S8HUdVF+V23yTvUNgz4Q+UoGs4sHSDEsIBFqNvInnpUmtNgcR2L5PAgMB
AAGjUzBRMB0GA1UdDgQWBBTPo8kfze4P9EgxNuyk7+xDGFtAYzAfBgNVHSMEGDAW
gBTPo8kfze4P9EgxNuyk7+xDGFtAYzAPBgNVHRMBAf8EBTADAQH/MA0GCSqGSIb3
DQEBCwUAA4ICAQAKHomtmcGqyiLnhziLe97Mq2+Sul5QgYVwfx/KYOXxv2T8ZmcR
Ae9XFhZT4jsAOUDK1OXx9aZgDGJHJLNEVTe9zWv1ONFfNxEBxQgP7hhmDBWdtj6d
taqEW/Jp06X+08BtnYK9NZsvDg2YRcvOHConeMjwvEL7tQK0m+GVyQfLYg6jnrhx
egH+abucTKxabFcWSE+Vk0uJYMqcbXvB4WNKz9vj4V5Hn7/DN4xIjFko+nREw6Oa
/AUFjNnO/FPjap+d68H1LdzMH3PSs+yjGid+6Zx9FCnt9qZydW13Miqg3nDnODXw
+Z682mQFjVlGPCA5ZOQbyMKY4tNazG2n8qy2famQT3+jF8Lb6a4NGbnpeWnLMkIu
jWLWIkA9MlbdNXuajiPNVyYIK9gdoBzbfaKwoOfSsLxEqlf8rio1GGcEV5Hlz5S2
```

```
    Start Time: 1792952392
    Timeout   : 7200 (sec)
    Verify return code: 18 (self-signed certificate)
    Extended master secret: no
    Max Early Data: 0
---
read R BLOCK
pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7
Correct!
kS0Hf0u5HiXFwKMKFqXvPdOTNGGa0X8V

closed
bandit15@bandit:~$ ls
bandit15@bandit:~$ ls -a
.  ..  .bandit14.password  .bash_logout  .bashrc  .profile
bandit15@bandit:~$ ^C
bandit15@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.

┌──(root㉿kali)-[~]
└─# ssh bandit16@bandit.labs.overthewire.org -p 2220
```

Level 17: Finding the Correct Service Port

Objective: Identify the listening port in the 31000–32000 range that provides the expected TLS service.

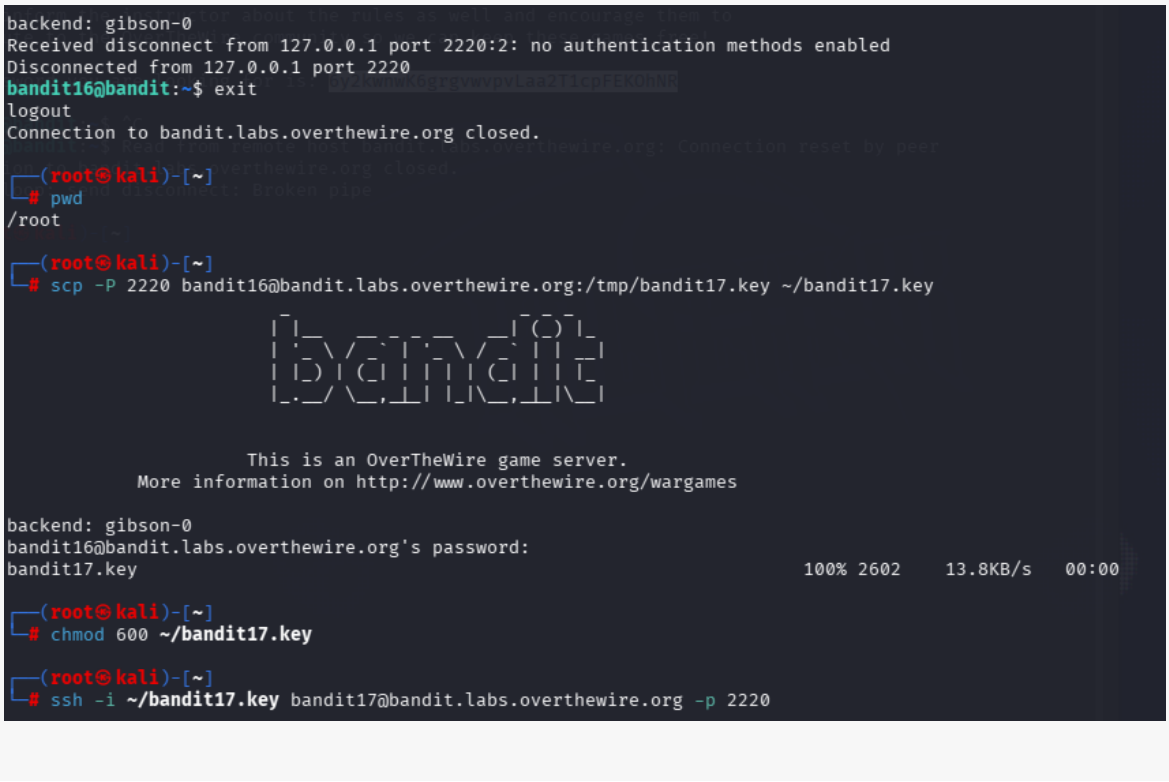
Commands / procedure:

```
nmap -sV -p 31000-32000 localhost
openssl s_client -connect localhost:<port> -quiet
```

Explanation: Nmap scans the specified port range and attempts to identify services. Test the candidate ports using the connection method indicated by the challenge. The correct service accepts the current password and returns an SSH private key for the next account.

Important note: Replace <port> with a port identified by the scan. Do not assume a port without testing it.

Password for bandit17: _____



Level 18: Comparing Password Lists

Objective: Compare the two password files to identify the new password.

Commands / procedure:

```
ls -la
diff passwords.old passwords.new
```

Explanation: diff reports differences between two text files. The challenge provides two files with nearly identical contents; the changed line contains the password for the next level.

Password for bandit18: OQxXZjELndr90zuhOTDYBEomI0SZITXI

```
bandit17@bandit:~$
bandit17@bandit:~$
bandit17@bandit:~$ diff passwords.old passwords.new
42c42
< pmTDnZLl0kLXTAU71zRbREj4tnzETcSP
---
> OQxXZjELndr90zuhOTDYBEomI0SZITXI
bandit17@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.

┌──(root㉿kali)-[~]
└─# ssh bandit18@bandit.labs.overthewire.org -p 2220
```

Level 19: Logging In Despite a Restricted Shell

Objective: Retrieve the readme contents even though the login shell exits immediately.

Commands / procedure:

```
ssh bandit18@bandit.labs.overthewire.org -p 2220 cat readme
```

Explanation: The remote command is supplied directly after the SSH connection details. This runs cat readme on the remote host without requiring an interactive shell, allowing the file to be read despite the shell restriction.

Important note: Enter the password for bandit18 when SSH requests it.

Password for bandit19: KpsOfPkcP7i1FlIExk2QEjyt6dw8dxZI

```
┌──(root㉿kali)-[~]
└─#     ssh bandit18@bandit.labs.overthewire.org -p 2220 cat readme
                         _                     _ _ _
                        | |__   __ _ _ __   __| (_) |_
                        | '_ \ / _` | '_ \ / _` | | __|
                        | |_) | (_| | | | | (_| | | |_
                        |_.__/ \__,_|_| |_|\__,_|_|\__|


                      This is an OverTheWire game server.
            More information on http://www.overthewire.org/wargames

backend: gibson-0
bandit18@bandit.labs.overthewire.org's password:
KpsOfPkcP7i1FlIExk2QEjyt6dw8dxZI
```

Level 20: Using a Setuid Helper Program

Objective: Use the provided helper to run a command with the permissions of its owner.

Commands / procedure:

```
ls -l
./bandit20-do cat /etc/bandit_pass/bandit20
```

Explanation: The helper program is configured to execute a command with elevated privileges for this challenge. Passing cat and the password-file path as arguments displays the password for bandit20, which is needed for the next level.

Important note: Only use the helper in the Bandit environment.

Password for bandit20: 4pIjcunZ0fK2vmp3IwfG8Vf7VhxD6pOA

```
bandit19@bandit:~$ ./bandit20-do
Run a command as another user.
  Example: ./bandit20-do whoami
bandit19@bandit:~$ ./bandit20-do whoami
bandit20
bandit19@bandit:~$ ./bandit20-do cat /etc/bandit_pass/bandit20
4pIjcunZ0fK2vmp3IwfG8Vf7VhxD6pOA
bandit19@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.

┌──(root㉿kali)-[~]
└─# ssh bandit20@bandit.labs.overthewire.org -p 2220
```

Level 21: Exchanging Data Between Two Connections

Objective: Use the suconnect program and a listening network connection to obtain the next password.

Commands / procedure:

```
# Terminal 1 (logged in as bandit19):
nc -lv 1234
# Terminal 2 (logged in as bandit20):
./suconnect 1234
```

Explanation: The listening netcat process waits for a connection on port 1234. The suconnect program connects to that port and checks the password supplied through the connection. When the current Bandit20 password is sent correctly, the program returns the password for bandit21. Keep the listener and client running in separate terminal sessions.

Important note: The listener must be running before the client connects. Use the port specified in your own lab session if it differs.

Password for bandit21: 4pIjcunZ0fK2vmp3IwfG8Vf7VhxD6pOA

```
bandit20@bandit:~$ ./suconnect 1234
        Read: 4pIjcunZ0fK2vmp3IwfG8Vf7VhxD6pOA
Password matches, sending next password
bandit20@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.

┌──(root㉿kali)-[~]
└─# ssh bandit21@bandit.labs.overthewire.org -p 2220
```

5. Skills Developed

- Connecting to a remote Linux server with SSH and specifying a port.
- Navigating directories and inspecting hidden files and file metadata.
- Handling unusual filenames and filenames containing spaces.
- Searching for files by size, owner, group, and type.
- Filtering and comparing text with grep, sort, uniq, and diff.
- Extracting printable strings and decoding Base64 and ROT13 data.
- Identifying file formats and unpacking nested compressed data.
- Using SSH private keys and understanding basic key-based authentication.
- Connecting to TCP services with netcat and establishing TLS sessions with OpenSSL.
- Scanning a limited port range with Nmap in an authorized training environment.
- Understanding setuid helper programs, restricted shells, and service-to-service communication.
- Recording evidence clearly and following an organized troubleshooting process.

6. Conclusion

The Bandit wargame provides practical experience with core Linux and introductory cybersecurity concepts. Completing the challenges encourages careful reading, systematic investigation, and the selection of suitable command-line tools for each task. The exercises also demonstrate how filenames, permissions, file formats, remote connections, and background services affect everyday system administration and security work.

The password fields and screenshot areas in this report are intentionally blank for completion with evidence from the student's own session. Any result that has not been verified in the lab should remain unfilled until it is confirmed. The skills practiced here form a foundation for further study of Linux administration, networking, and authorized security testing.